

Biocontrol modeling project: Environment Setup and Installation Manual

Mateo Franco

March 30, 2026

Contents

1	Introduction	3
2	Option A: Local Installation	3
2.1	Python Installation	3
2.1.1	Step 1: Access to the official download page	3
2.1.2	Step 2: Download the installer	3
2.1.3	Step 3: Execute the installer	4
2.1.4	Step 4: Check the installation	4
2.2	Installing Visual Studio Code	5
2.2.1	Step 1: Access to the official website	5
2.2.2	Step 2: Download the installer	5
2.2.3	Step 3: Run the installer	6
2.2.4	Step 4: Launch VS Code	6
2.3	Cloning the GitHub Repository	7
2.3.1	Step 1: Sign in to GitHub in VS Code	7
2.3.2	Step 2: Clone the repository	7
2.3.3	Step 3: Confirm the project was cloned	9
2.4	Creating and Activating the Virtual Environment	10
2.4.1	Step 1: Open the built-in terminal in VS Code	10
2.4.2	Step 2: Create the virtual environment	11
2.4.3	Step 3: Activate the virtual environment	11
2.4.4	Step 4: Select the Python interpreter in VS Code	11
2.4.5	Step 5: Install the required dependencies	12
3	Option B: Docker Installation	12
3.1	Why use Docker	12
3.2	Install Docker Desktop	13

3.3	Project Configuration	13
3.3.1	Dockerfile	13
3.3.2	docker-compose.yml	14
3.4	Running the application	15
3.5	Stopping the application	16
3.6	Viewing Logs (Troubleshooting)	16

1 Introduction

This guide provides a step-by-step visual tutorial to help you set up your development environment and run the project locally. Includes screenshots and detailed instructions for each step.

By the end of this guide, you will have:

- Installed Python and Visual Studio Code.
- Cloned the project repository directly from Visual Studio Code.
- Created and activated a virtual environment.
- Installed all the required Python libraries.

Make sure to follow each step carefully and verify that the output matches the screenshots provided.

2 Option A: Local Installation

2.1 Python Installation

Python is the programming language used in this project. In this guide, we will use Python version 3.10.9, which is required for compatibility with the project dependencies.

Here is how to install it correctly on your computer.

2.1.1 Step 1: Access to the official download page

Go to the official Python page using the following link:

<https://www.python.org/downloads/windows/>

2.1.2 Step 2: Download the installer

And look for Python version 3.10.9 (64-bit)

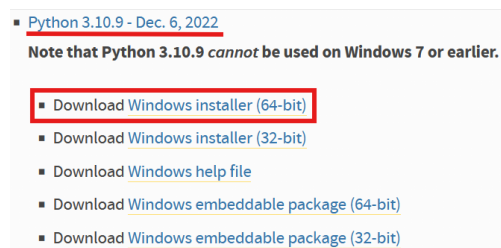


Figure 1: Python download official page.

2.1.3 Step 3: Execute the installer

When the download file is executed, the following window will appear.

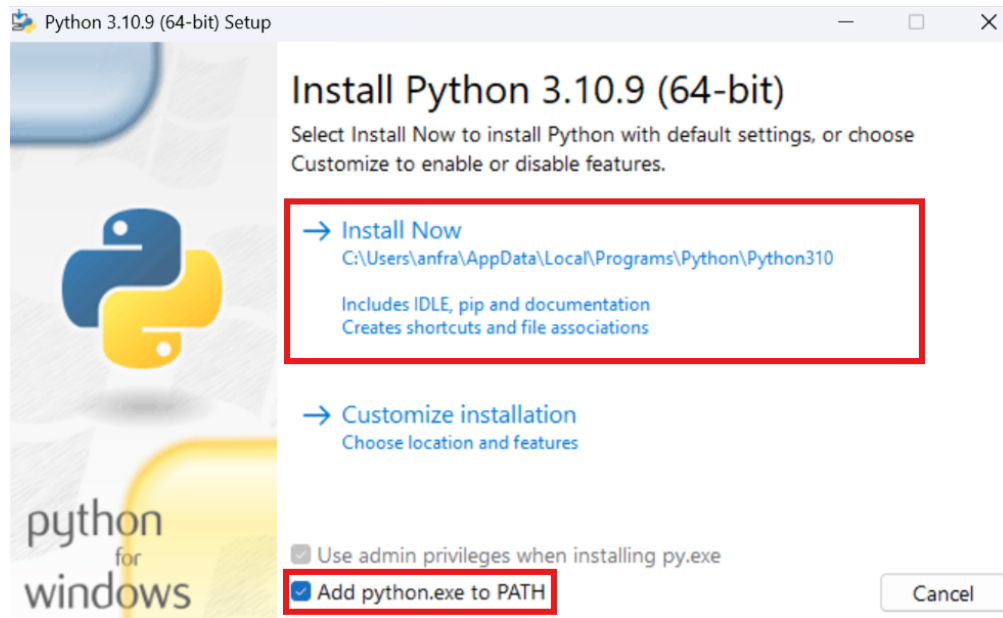


Figure 2: Python installer window.

1. Check the box that says "**Add python.exe to PATH**" (is essential for it to work from the terminal).
2. Then click on "**Install Now**" to start the installation.

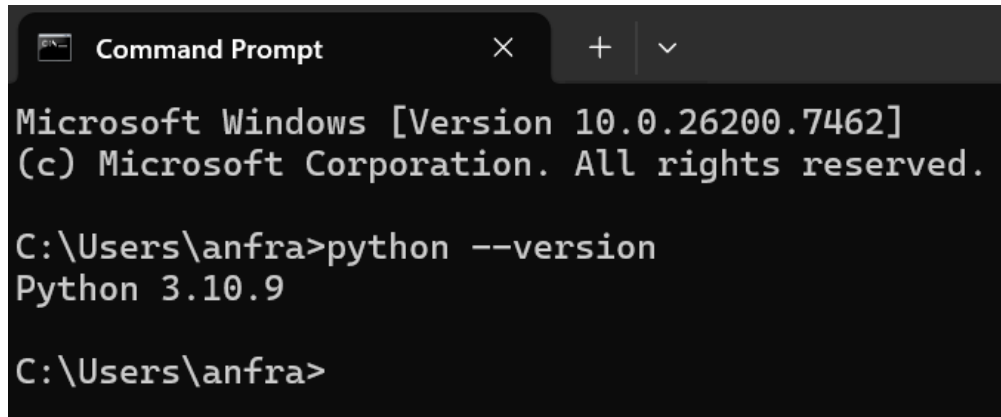
2.1.4 Step 4: Check the installation

Once the installation is complete, open the terminal (or cmd) and type the following command. You can type windows key + R to open the terminal.

```
python --version
```

If everything is correct, you will see the following message:

```
Python 3.10.9
```



```
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anfra>python --version
Python 3.10.9

C:\Users\anfra>
```

Figure 3: Command prompt showing the installed Python version.

If a different version appears, you may have multiple Python versions installed. In that case, make sure to use Python 3.10 when creating the virtual environment.(See 2.4)

2.2 Installing Visual Studio Code

Visual Studio Code (VS Code) is a lightweight and powerful code editor that supports Python development through extensions. In this section, you will learn how to download and install VS Code on your system.

2.2.1 Step 1: Access to the official website

Go to the official Visual Studio Code website using the following link:

<https://code.visualstudio.com/>

2.2.2 Step 2: Download the installer

The homepage should look like the following image:

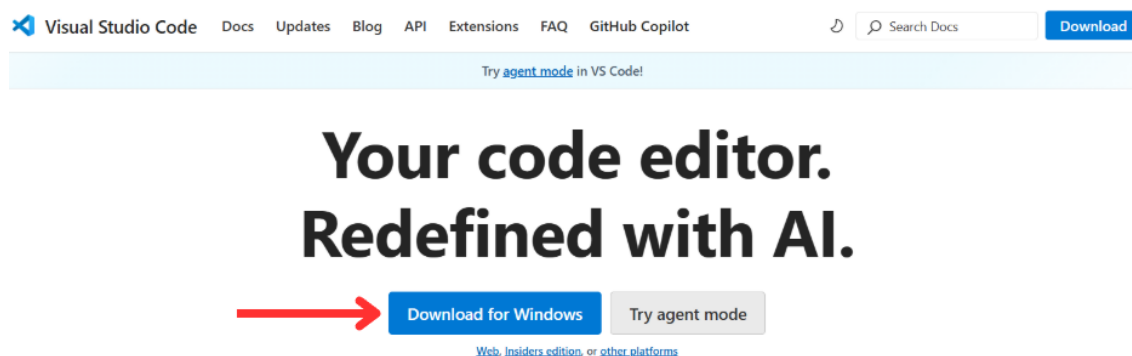


Figure 4: VS Code download page for Windows.

1. Click on the **"Download for Windows"** button to download the installer.

2.2.3 Step 3: Run the installer

After downloading, open the installer file. The setup wizard will appear:

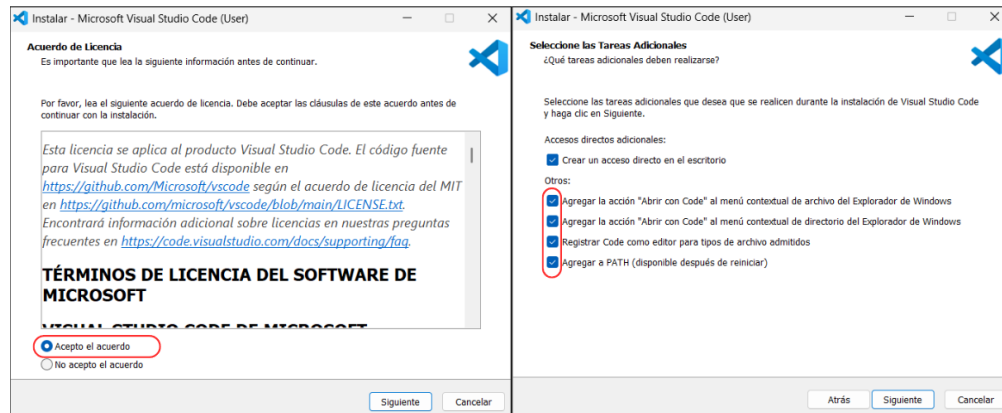


Figure 5: VS Code installation setup window.

1. Accept the license agreement and click **"Next"**.
2. Choose the installation folder or leave the default, then click **"Next"**.
3. In the "Select Additional Tasks" step, make sure to check:
 - **"Add to PATH"**
 - **"Register Code as an editor for supported file types"**
 - **"Add 'Open with Code' action to Windows Explorer"**
4. Click **"Install"** to begin the installation.

2.2.4 Step 4: Launch VS Code

Once the installation is complete, you can open Visual Studio Code from the Start Menu or by typing:

`code`

in the command prompt (only if you added it to the PATH during setup).

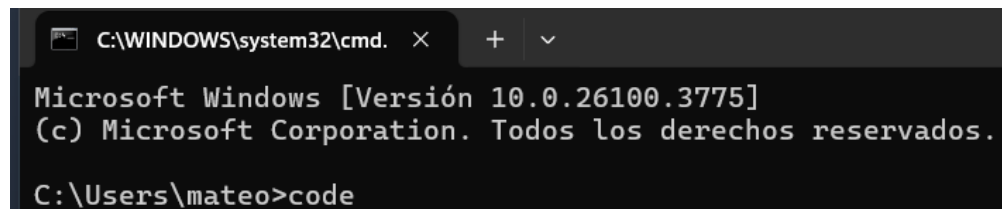


Figure 6: Opening VS Code from the terminal using `code`.

2.3 Cloning the GitHub Repository

Before you can clone the project repository, you need to have:

- Git installed on your system (see previous section).
- A GitHub account already created.
- You must be signed in to GitHub within Visual Studio Code.

2.3.1 Step 1: Sign in to GitHub in VS Code

Open Visual Studio Code. On the bottom-left corner, click on the account icon and select “**Sign in to GitHub**” or “**Sign in to GitHub to use GitHub copilot**”.

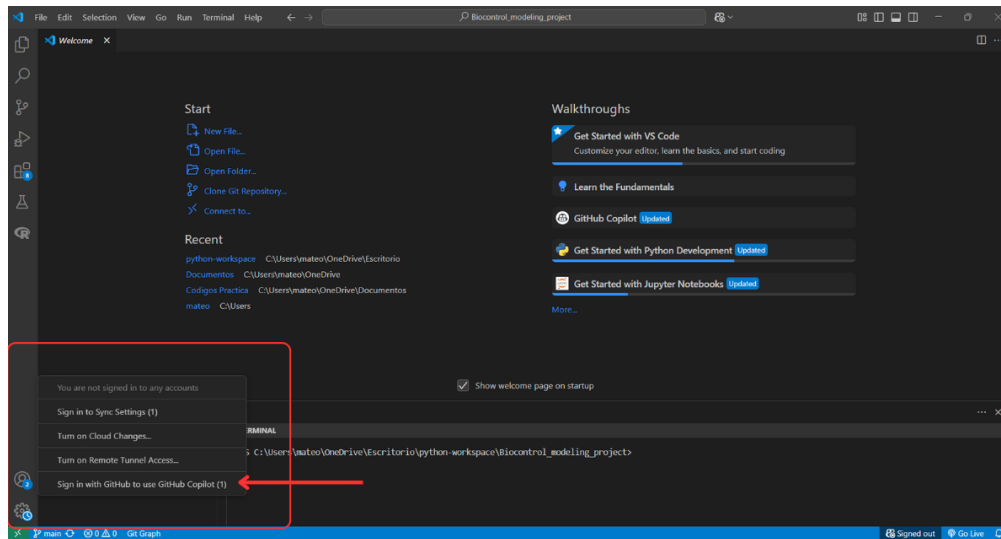


Figure 7: Signing in to GitHub in VS Code.

Follow the prompts to complete the login process.

2.3.2 Step 2: Clone the repository

- Click on file section, then click on new window

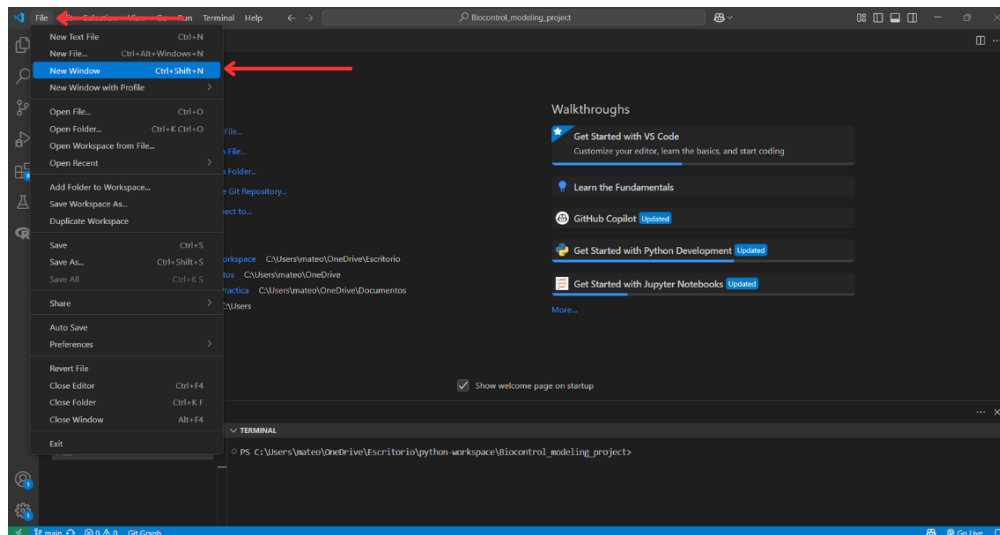


Figure 8: Accessing the new window option in VS Code.

- Click on **Clone Git Repository...**

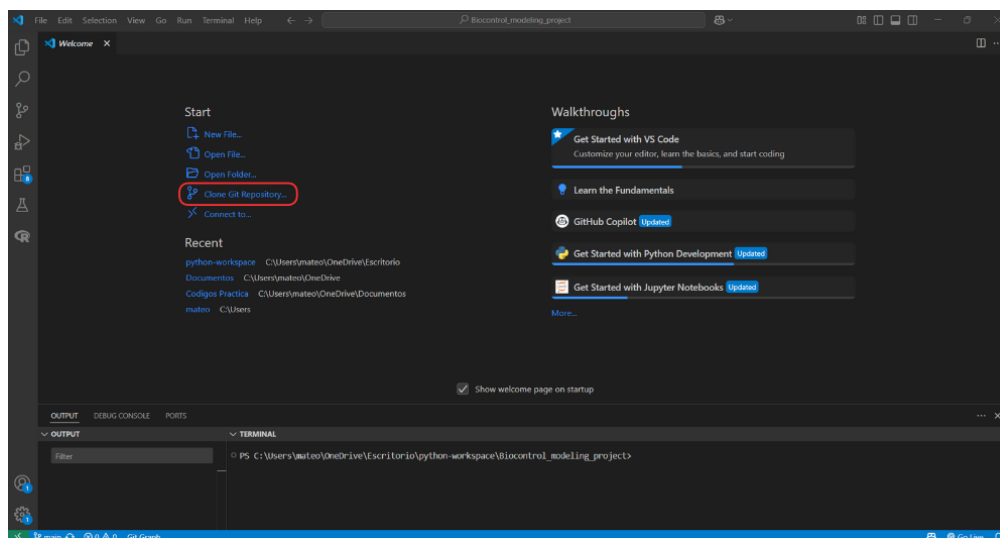


Figure 9: Accessing to Clone Git Repository window.

- Paste the repository URL in the input box that appears at the top.

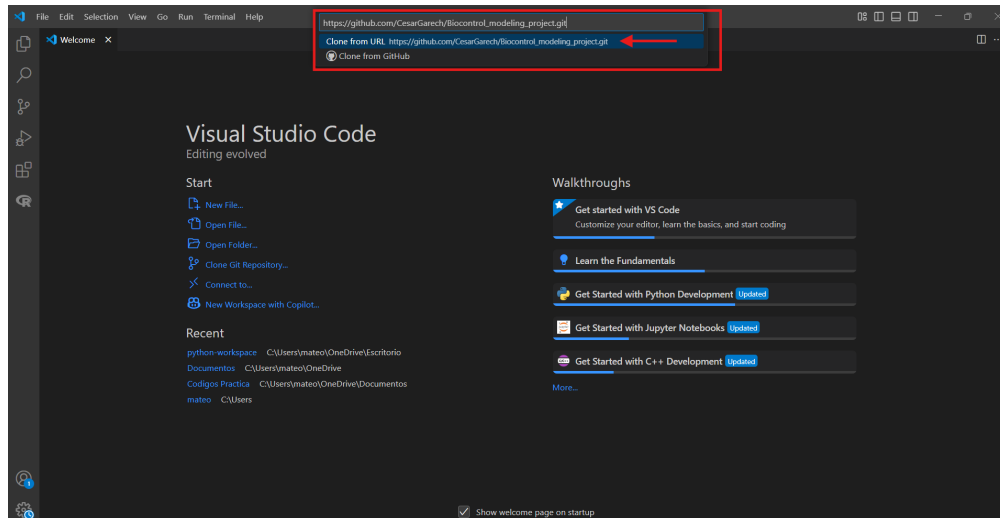


Figure 10: Copy the url repository.

- Choose a folder on your local machine where the repository will be cloned.

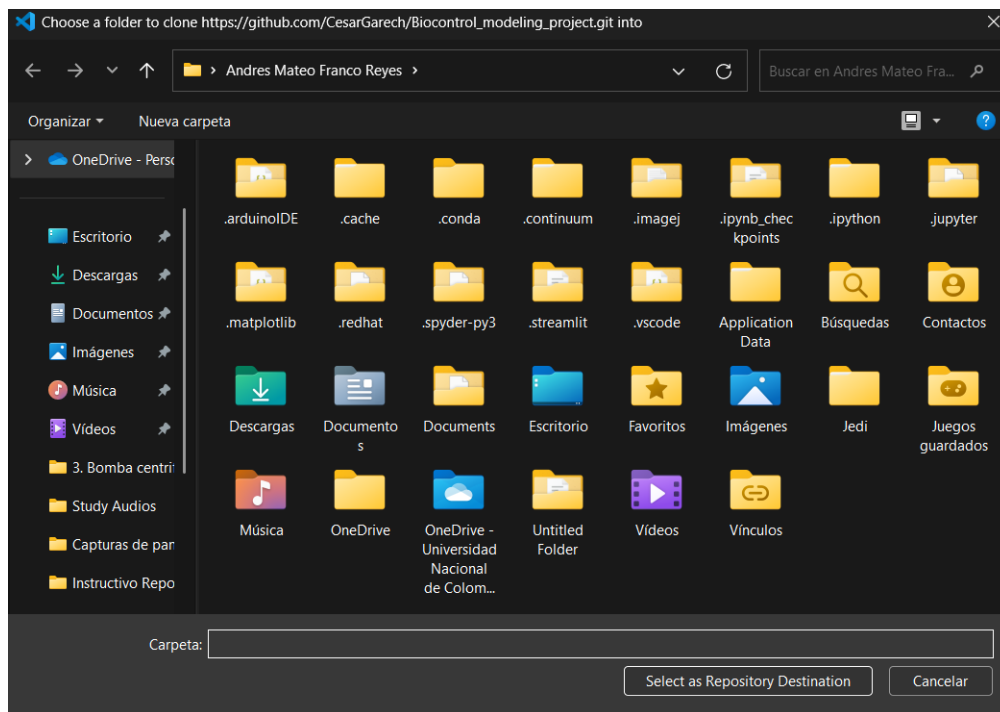


Figure 11: Choose a folder to cloned the repository.

2.3.3 Step 3: Confirm the project was cloned

Once you finish to choose a destination folder, you should see the project structure on the left panel (Explorer view). You can now start working with the code

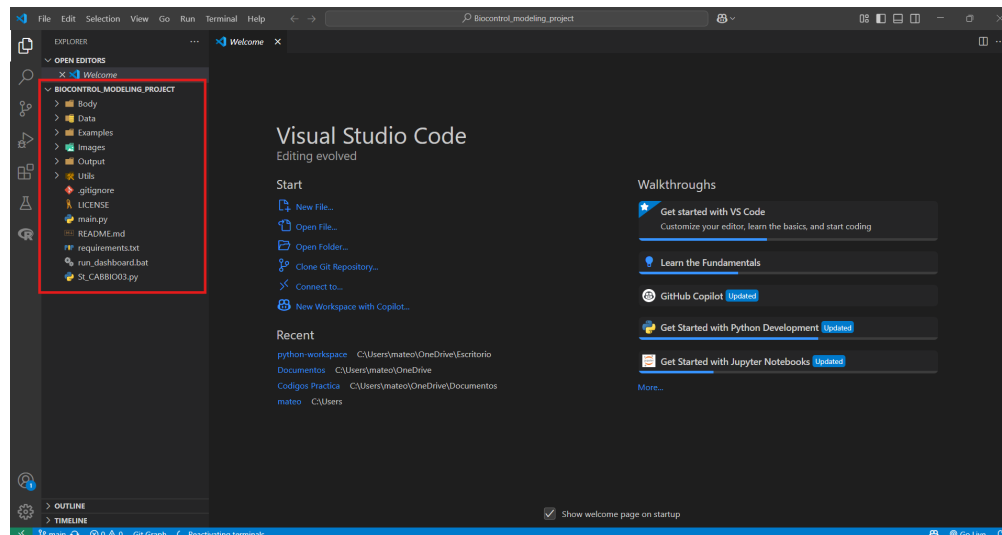


Figure 12: Repository cloned.

2.4 Creating and Activating the Virtual Environment

To isolate the project dependencies and avoid conflicts with global packages, it is recommended to use a virtual environment.

2.4.1 Step 1: Open the built-in terminal in VS Code

With the project opened in Visual Studio Code, go to **Terminal > New Terminal** to open the built-in terminal.

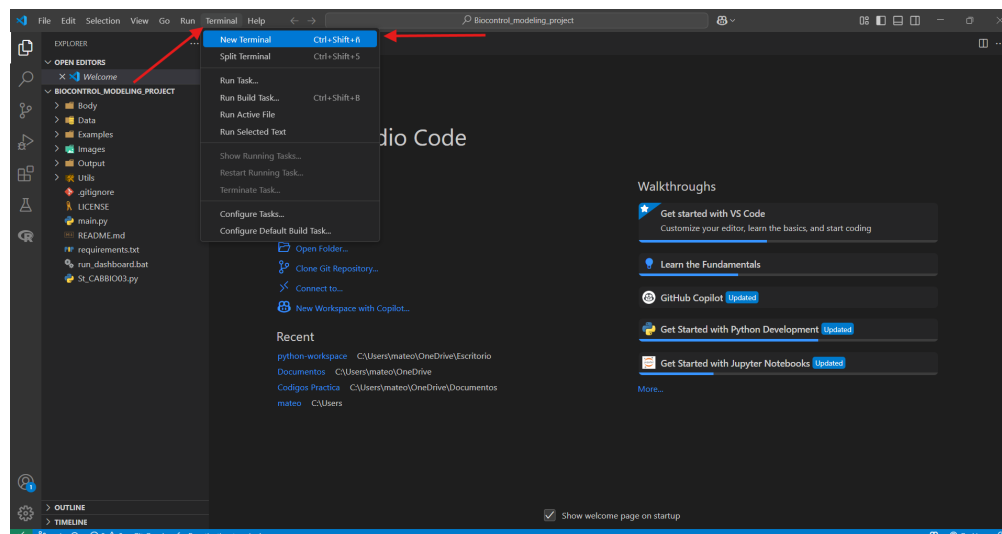


Figure 13: Opening the integrated terminal in VS Code.

2.4.2 Step 2: Create the virtual environment

If you have multiple Python versions installed, it is important to explicitly use Python 3.10 to create the virtual environment.

In the terminal, type the following command and press Enter:

```
py -3.10 -m venv venv
```

This will create a folder named 'venv' containing the virtual environment using Python 3.10.

After creating the environment, you can verify the Python version with:

```
venv\Scripts\python --version
```

You must see Python 3.10.9.

2.4.3 Step 3: Activate the virtual environment

Depending on your operating system, use the appropriate command:

On Windows:

```
venv\Scripts\activate
```

On macOS / Linux:

```
source venv/bin/activate
```

Once activated, you will see the environment name ('venv') in parentheses at the beginning of the terminal line.

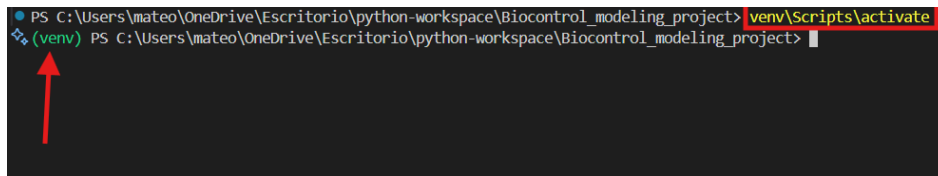


Figure 14: Virtual environment activated in the terminal.

2.4.4 Step 4: Select the Python interpreter in VS Code

To ensure Visual Studio Code uses the correct Python version:

1. Press Ctrl + Shift + P
2. Type: Python: Select Interpreter
3. Select the interpreter located in:

```
venv\Scripts\python.exe
```

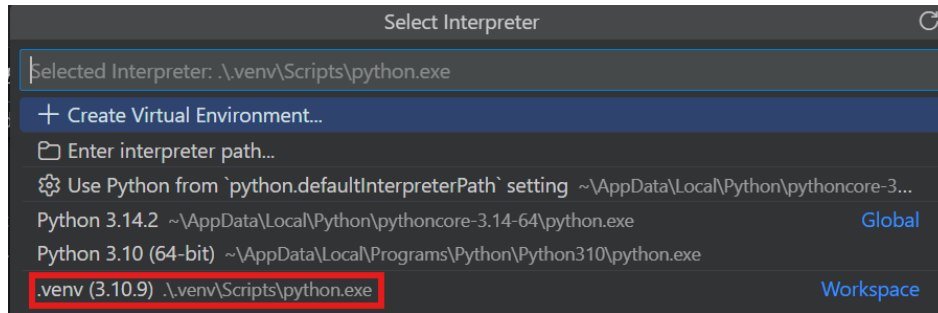
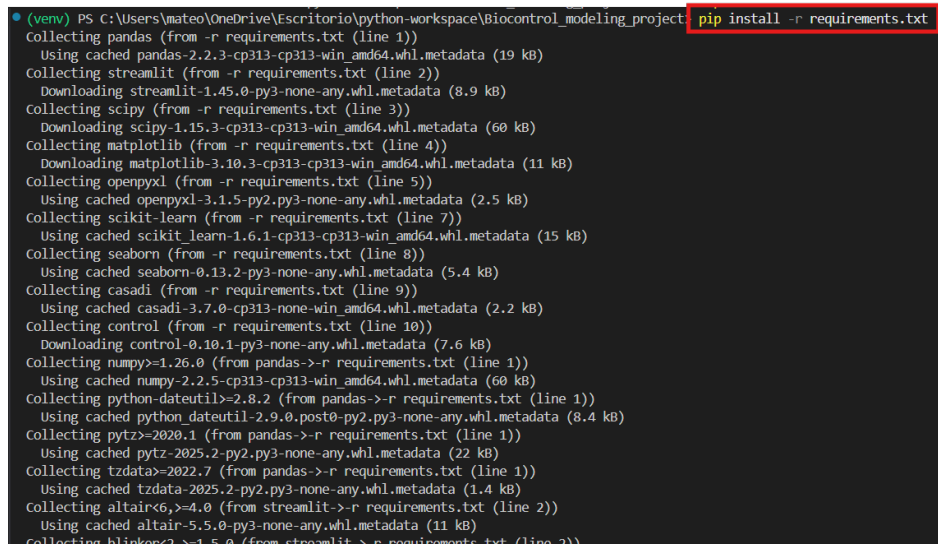


Figure 15: Selecting interpreter

2.4.5 Step 5: Install the required dependencies

In your project includes a `requirements.txt` file, run the following command to install the required libraries:

```
pip install -r requirements.txt
```

Figure 16: Installing the dependencies listed in `requirements.txt`.

3 Option B: Docker Installation

3.1 Why use Docker

Docker is a containerization platform that allows applications to run in isolated, reproducible environments. Instead of manually installing dependencies such as Python libraries, compilers, or LaTeX distributions, Docker packages everything into a single container that behaves identically across all systems.

For the Bioprocess Control Simulator, Docker provides the following advantages:

- **Reproducibility:** Ensures consistent numerical results when using advanced optimization and estimation tools such as NMPC, EKF, and RTO.
- **Dependency Isolation:** Avoids conflicts between system libraries and Python packages (e.g., CasADi, SciPy, PyTorch).
- **Integrated LaTeX Support:** Automatically compiles .tex files into PDF reports without requiring local LaTeX installation.
- **Simplified Deployment:** Enables easy execution on cloud platforms (e.g., AWS, Render, Hugging Face Spaces).

Important: Docker replaces the need for a local Python installation and virtual environment.

3.2 Install Docker Desktop

Follow these steps to install Docker:

1. Go to the official Docker website: <https://www.docker.com/products/docker-desktop/>
2. Download the installer for your operating system:
 - (a) Click on "Download Docker Desktop"
 - (b) Download the **Windows (AMD64)** installer
3. Run the installer and ensure the option **Use WSL 2 instead of Hyper-V** is selected
4. Restart your computer if required
5. Log in into Docker
6. Open Docker Desktop and:
 - (a) Sign in to Docker
 - (b) Verify the status indicator shows "**Engine Running**"

3.3 Project Configuration

Before running the application, ensure that the following files exist in the root directory of the repository:

3.3.1 Dockerfile

This file defines the environment, installs OS-level dependencies (like LaTeX), and sets up Python.

```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 RUN apt-get update && apt-get install -y \
6     build-essential \
7     texlive-latex-base \
8     texlive-fonts-recommended \
9     texlive-fonts-extra \
10    texlive-latex-extra \
11    && rm -rf /var/lib/apt/lists/*
12
13 COPY requirements.txt .
14
15 RUN pip install --no-cache-dir -r requirements.txt
16
17 COPY . .
18
19 EXPOSE 8501
20
21 CMD ["streamlit", "run", "main.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

Listing 1: Dockerfile

3.3.2 docker-compose.yml

This file defines how the container is built and executed.

```
1 version: '3.8'
2
3 services:
4   biocontrol_app:
5     build: .
6     container_name: Process_Simulator
7     ports:
8       - "8501:8501"
9     volumes:
10      - ./app
11     environment:
12       - PYTHONUNBUFFERED=1
13       - STREAMLIT_SERVER_ENABLE_CORS=false
14       - STREAMLIT_SERVER_ENABLE_XSRF_PROTECTION=false
15       - STREAMLIT_SERVER_HEADLESS=true
```

Listing 2: docker-compose.yml

3.4 Running the application

Open your VS Code terminal (or Git Bash) in the repository folder and use the following commands:

- Step 1: To Build and Start the Simulator (Background Mode):

```
docker compose up -d
```

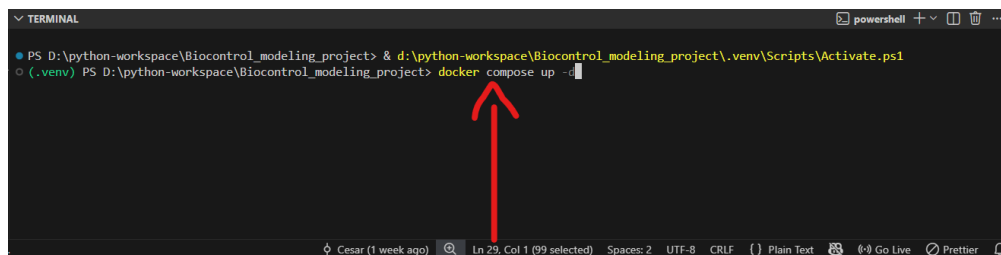


Figure 17: Build Container.

This step could take several time

- Step 2: Access the Application:
Open Docker application and run the container:

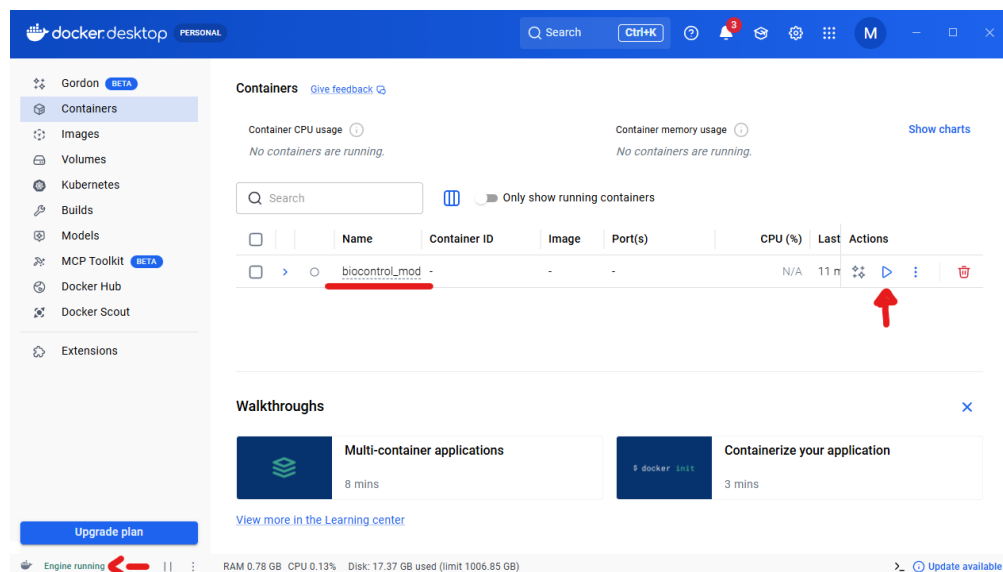


Figure 18: Docker Application.

- Step 3: Open your browser and go to:

`http://localhost:8501`

- Step 4: Enjoy the application

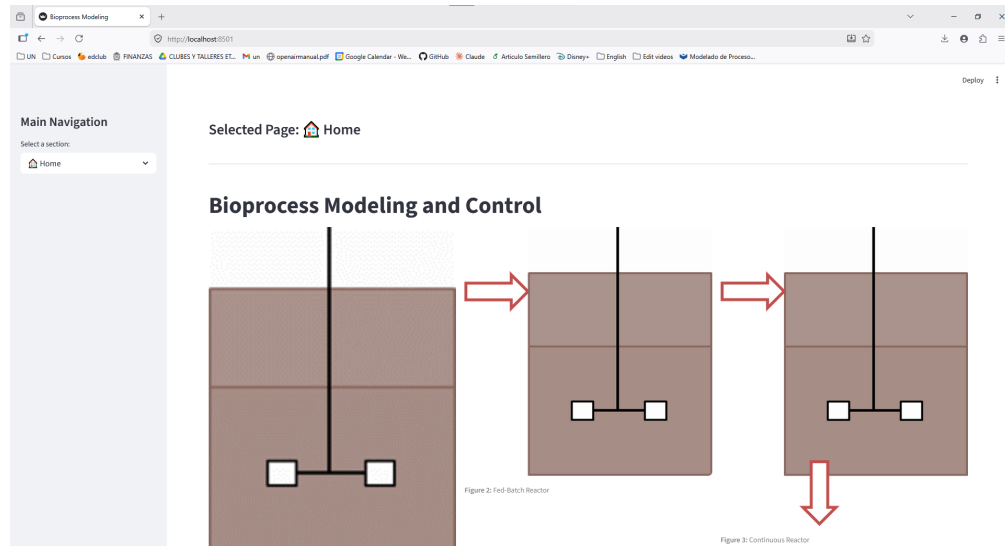


Figure 19: Application Window.

3.5 Stopping the application

To stop the container without deleting it, write this in the terminal:

```
docker compose stop
```

3.6 Viewing Logs (Troubleshooting)

If your Python code has an error, you can check the terminal output by running:

```
docker compose logs -f
```